CARLA Monocular Depth Estimation:

SL-3



Acey Dufrensne

CS 4850 section 01

(404-889-2819)

Index:

1.0 The Technical Narrative 3

- 1.1: Revised Goals 3
- 1.2: Dataset Implementation 3
- 1.3: Encoder Implementation 4
- 1.4: Mathematical Implementation 6
- 1.5 Model Architecture Implementation 7
- 1.6: Tokenizer Implementation 8

2.0 Data and Connectivity 8

3.0 Setup and Deployment Guide 9

The Technical Narrative:

The initial implementation of this project began with research, specifically locating the current best depth estimation models, then extracting areas in which these models could use some form of innovation or improvement.

- **Revised Goals:**

The primary goals remain primarily unsullied from the design phase of the project, however the scope has been tightened. Specifically, this paper is focused on two research projects, the 'SAM3D' model by Meta and the Axelotl3D model by Nvidia. In particular, we will focus on maintaining the accuracy of these models, while hopefully improving efficiency and speed while generating a prediction. An important, which will influence the implementation choices, is that Axelotl3D makes use of partial point cloud input, which heavily improved it's resulting accuracy and findings within the corresponding paper. I have discussed this with my mentor, and we've agreed that this is not necessarily in the spirit of monocular depth estimation, as starting with a three-dimensional 'starting point' seems relatively unfair, as SAM3D did not utilize this, and by the very definition, depth estimation is from two-dimensional input, to three-dimensional output. As such, while we do make use diffusion, we opted to obtain the initial noise by a more traditional approach, thus starting with two-dimensional input.

The two main papers selected were the research works on the Axelotl3D model from Nvidia, and the SAM3D model from Meta. These papers provided a good format and progression for the benchmarks that CARLA should reach, as well as a useful starting point for plotting the architecture designs.

The initial goal was to discover three specific components or design choices that could be changed and improved, those being the model encoder, the main algorithm, and an improved and expanded dataset. The primary goals, as resources are limited, were to be able to run the model on commercial grade hardware, as well as compare benchmark testing against the Nvidia and Meta models on the same equipment.

(As this section has been discussed in the previous project documentation, I have elected to only briefly recount how the dataset is built).

- **Dataset Implementation:**

The project started with setting up the datasets, which were a cultivation of the Objaverse-XL dataset, which contains models sourced from four separate directories: the Smithsonian, and various Github repositories. Then gave a strong general base for the model to learn from and included a wide range of everyday items. Each mesh, or model, was rendered in Blender version 3.3.0. This script utilized the BPY, or Blender Python API. Blender uses a virtual environment,

the script initializes a camera object, and renders images of each model in a layered ring pattern.

This means the final dataset is a set composed of one GLB file, with a corresponding twelve images for each individual mesh. The GLB file contains the three-dimensional coordinates for each model, the faces, which group the cluster of three coordinates sets that combine into a triangle, the normals, the UV coordinates, which provide the texture location and information, and the textures, which is what the UV map uses to create a topology and color.

On top of this set, I wanted the model to be specialized on human topology and geometry as well, which would help meet the criteria of the goals specified in the design documentation. To accomplish this, I used the MakeHuman API, version 1.2.0, and a 3.13.5 Python script to generate randomized human characters. This involved creating a pipeline of randomized ratios. The script starts with initializing a uniform randomized value for the head height, which is determined based on an age variable. This head height is then used to generate the rest of the proportions of the body. These models are then exported into the BPY script, and are processed the same as the Objaverse-XL set.

- **Encoder Implementation:**

One of the main contributions of this research is a new way of improving on the ideas of Axelotl3D with a different approach to classification. In Nvidia's paper, and the original design specifications of this research, a separate multi-class classification model was proposed, however after reviewing the results of many modern generative depth estimation models, it was recognized that using the same approach would lead to the same issues as many of the current model peers.

Specifically, no matter how well trained these classifiers are, they seem to miss some amount of objects within an image. To change this approach, I decided to borrow some of the ideas behind VL-Jepa, which a training architecture used by many identification models, which works by partially obstructing images in the training set, and training the model to correctly identify objects based on partial vision. In accordance with the goals of the research, I felt it would be a fair trade-off to attempt a heuristic approach, to save computational time by reducing the amount of models in the final algorithm.

To do this, we start with a Sobel Edge detection, which was chosen over the Canny Edge detection, as well as the Hough edge detection. The Canny Edge detection which is objectively a more in depth algorithm, which is useful to reduce noise and produce a higher quality edge map. In this case Sobel was opted over this algorithm, as all the images were rendered, so noise was not an issue, and any additional benefits came at the cost

of a slightly higher computational cost of the algorithm, which adds up when the dataset contains over half a million models.

The Hough Edge detection was very seriously considered, as will become apparent shortly. Hough is not a normal edge detection algorithm, and is used to represent an edge mathematically. This concept would be imitated, however, in the final script. After an edge map has been created using the Sobel algorithm, we then transform the pixel maps into mathematical functions. This is a simple process, where we start with the edge map produced by Sobel, we then use the OpenCV Skeletonize function, which reduces the thickness of all edges in the map.

Then we can start with first pixel in the edge map array, check the eight surrounding 'neighbor' pixels and check if any are also edge pixels. Assuming one is, we can use the very simple function: y = mx + b, where 'm' is our slope, and 'b' is our y-intercept. Then we can continue this process of adding a neighboring pixel into our linear function, using an accuracy threshold of seventy-five percent and an offset distance threshold of .05, or five percent of the image width and height, we can ensure that each of these pixels belong to the same line. Once we have encountered a pixel that no longer fits these parameters, we create a new line and repeat the process of merging edges together, until we have a plotted graph of functions, or a linear system.

However, an issue soon becomes apparent, for example, take a perfect circle, using this method the perfect final linear system would have radius(1-cos(pi / n), where n represents how many segments would pass the algorithms accuracy and distance check, linear functions to represent it. This is acceptable in most circumstances, however in accordance with the stated goals, we want to optimize these representations as correctly as possible.

We can take this a step further, by introducing arc length of our parametric curve. To do this, we can use the same idea of an iterative process, this time traversing through functions, where we have line one as x(t), and line two as y(t), we can use a Bezier curve, which utilizes four control points, where the endpoints are determined by the bounds of the first pixel in x(t), and the last pixel in y(t). Then just like a network, we can adjust the center two points, and repeatedly check the accuracy until we pass the distance and accuracy check.

After these iterative processes have been completed, we can search for any edges that are less than 10 points, or ten connective pixels in length, and break them down to their base pixels. Then we may check those against all edges that are longer than that threshold, and compare if any of the pixels belonging to a

'short' function can be merged into a 'long' function.

It is important to note that all these calculations are done in the first quadrant of the coordinate plane, this will come up in later sections.

- **Why do we need mathematical representations of our models in a two dimensional space?**

This seems contradictory to a model that should be operating within a three-dimensional space, but it is not. A depth estimation takes a two-dimensional input and provides a three-dimensional output, therefore we need a solid two-dimensional base understanding for the model, to give it the most complete and accurate input information to inform it' s final output prediction.

Now that we have all the dataset meshes converted into a system of linear functions, we can now apply basic linear algebra and a little calculus to compare inputs to our training set.

Recall from early Calculus courses, that integration is the measurement of continuous change. Further, we can call a constant scaling force k is not calculated in a manner that affects the base integral, f(x), such that f(x)dx is equivalent to k(f(x))dx, this will be explained the following section.

Now, we have a system of equations for every mesh, then we can keep the ideas of VL-JEPA, by partially obstructing our training data, and having the model attempt an identification of the object. In this case, however, the model is not passing this input through a neural network, but rather a simple heuristic approach, such that it takes the partially completed edge map, after processing and refining, and compares that incomplete edge map to the complete edge maps in the training set.

Here the algorithm compares distances between two bounds between two functions. This is a nested loop, which iterates first through each function in the system of equations of the input object, and comparing it to the systems of equations of the training set. This gives us $O(n^k)$, this is not an optimal approach, and completely negates the reasons for a heuristic progression.

Instead, we convert our systems of equations into a hash look-up. We create a bucket system, where similar functions are grouped by slopes and types, for example, consider quadratics and linear functions in two groups, and refer to the earlier section, where we defined both. A Bezier curve is defined by four weighted points, and a linear function by slope, again we are not considering scaling factors. So if the input function is: '$2x^2 + 3dx$' then we know to first move into the quadratic functions, then we can search by geometric properties, consider slope. To find slope we calculate the derivative of the input function, $f(x) = 2x^2+2\ dx == f'(x) = 4x$. Similar to weights in a neural

network, which act to adjust and modify input as it passes through a transformer, we instead reduce the scope of the final output by categorizing the geometric features of the functions.

Once we have reduced the amount of searches to a look-up, we obtain a value very close to O(1). At this point we have some amount of reduced objects that are in a range close to the input. We can then begin a more exact search to find the closest possible match.

Of course it is important to note that we are comparing an input that is obscured by some amount. This means even if an image from the training set is passed to the model, the most ideal circumstance, we still cannot have an absolute match. To counter this, we can apply a sliding gradient window on the selected proposed training objects, and return the top k objects.

This is for single object identification, to complete for a complex image, we use the same methodology, but instead, we apply the sliding window to both images, input and selected training images. As the algorithm 'finds,' more objects, we place a bound on that portion of the edge map, and continue until we have 'solved' the image, where every edge has been assigned. We can imagine this as a depth map, specifically that we have a bound of the lowest depth, essentially we understand that somethings will not be able to be classified, think of a background, for example the depth of field is too far.

In these cases, as mentioned, we may treat the background as an object in itself. Meaning that it is so far from the focus of the lens it does not need to be rendered. This means the encoder acts a large system function, where f(x) is a system of systems of equations of the training set, and x is a singular system of equations, then the encoder behaves like: '$(f(x) - g(x))^k$.'

- **Model Architecture:**

With the previous results in mind, we can begin training the main transformer model with the VL-JEPA architecture, using batch training. This is a form of shadowy reflection to how the encoder behaves, for intention.

When we train the model, we use the same bucket system as mentioned in the encoder. That is we train on the clusters grouped by function types as the primary feature, with some amount of randomized training objects inserted to prevent over-fitting. This is useful, as some images with similar edges will have similar three-dimensional volume and shaping, but some will not. This allows the model to then utilize the additional extracted features to help learn the differences between similar inputs.

While a transformer architecture will be the primary model design, the results of

the Nvidia research must be acknowledged, and integrated into the model for the best possible results. Their use of diffusion generation architecture is worth emulating. The main drawback of this design, is that the diffusion generation is much more computationally expensive than more traditional transformer architecture. With this being understood, we can then decide not to use this for all predictions, but only use it for it's strength, which is partially obscured objects. Mentioned in the desired goals, at the beginning of this paper, that Nvidias decision to start with a partial pointcloud object seemed to not be in spirit with the concept of monocular depth estimation, instead we can use the data generated by the encoder, as the model's starting point, and then introduce our noise to those coordinates.

- **Tokenizer:**

We still have not accounted for how we use the data processed by the encoder, and handed off to the diffusion network. We use a special tokenizer and the rely on the BPY again.

While we can, and do pass the complete three-dimensional data from the encoder, with the corresponding objects when edge matching between input and the training set are similar, we can refine this data as well.

We extract the most common edges in the training set, then bring those objects into the BPY, Blender Python API, we can then use ray tracing to find the immediate coordinates that represent the edge pixels, and return their relative depth, compared to the center of the model.

This gives us a good substitute for the point-clouds utilized by Axelotl3D. Then if the model finds a match, using the subtractive process discussed above in the encoder section, we can then predict that the object is partially obstructed, and this is when instead of passing this label classification to the transformer architecture, we instead hand it off to the diffusion model.

The diffusion model operates by using multiple passes over the input, rather than one large sweep like traditional transformers. During training, partial noise is added to the training set representation, which the model then learns how to put back together, similar to how GAN architecture works. The usefulness of diffusion comes from the architecture's ability to use pieces of the input to build the rest of the prediction.

For example, imagine an image of a lion is passed to the model, but it's back right leg is obscured. The model can use the data from the back left leg to influence it's prediction of the mesh. This is the part of the architecture we want, with limited use, to reduce the heavy computational cost.

Data & Connectivity:

n/a

Setup and Deployment Guide:

First download and install the following software:

- MakeHuman: v1.2.0
- Python Programming Language: v3.13.5
- Python Programming Language: v3.7.6
- GCC(C++ compiler): v16.2.0
- C++ Programming Language: vC++17
- Blender: v3.3.0
- Blender Python API: v3.3.0

Then run the ‘createModList.py’ script, adjust the file path variables in ‘createHuman.py’ script. Then run the script. Next run the ‘Startup.py' script, as well as the ‘presetData.py' script. Finally, run the ‘getAnimations.py' script.

This sets up and initializes the required datasets for the model. Next, download the ‘coord_decoder.py,’ script, then run the ‘renderer.py’ script, as well as the ‘renderHelper.py' script. This will download the physical rendered images from the three-dimensional dataset.

Finally, download the ‘source.cpp,’ ‘BPETokenizer.py,’ ‘processToken.py,’ and ‘edgeEncoder.cpp’ files and scripts. Then wait patiently until I have the main architecture built.

References:

yoshiyasu.shimizu.research@gmail.com, Yoshiyasu Shimizu Affiliation: [2pt] Independent researcher Affiliation: [2pt], and Affiliation: [2pt] Independent researcher Affiliation: [2pt] yoshiyasu.shimizu.research@gmail.com. “SWEEPLSD: A One-Pass, O(Width)o(\text{width})-Memory Line Segment Detector with an Integer-Only Streaming Core and a Real-Time FPGA Realization.” *SweepLSD: A One-Pass, O (Width) -Memory Line Segment Detector with an Integer-Only Streaming Core and a Real-Time FPGA Realization*, 22 Aug. 2026, arxiv.org/html/2608.22086v1.

Team, SAM 3D, et al. “Sam 3D: 3dfy Anything in Images.” *arXiv.Org*, 2 June 2026, arxiv.org/abs/2511.16624.

Hu, Anita, and Maria Shugrina. “Axolotl3D: A Unified Framework for Faithful 3D Shape Completion.” *Arxiv*, Nvidia, 22 July 2026, arxiv.org/pdf/2505.20660.

Chen, Delong, et al. “VL-JEPA: Joint Embedding Predictive Architecture for Vision-Language.” *Arxiv*, Meta FAIR, HKUST, Sorbonne University, NYU, 2 Feb. 2026, arxiv.org/pdf/2512.10942.